

Algorithmen und Datenstrukturen

Übung 3: Grundlagen

Robert

2026-05-22

Wiederholung Mastertheorem

Mastertheorem

- das Mastertheorem ist eine Methode, die auf rekursive Gleichungen der Form $T(n) = a \cdot T(\frac{n}{b}) + f(n)$ angewendet werden kann.
 - diese Gleichung beschreibt, wie ein Teile-und-Herrsche-Algorithmus ein Problem n in a Teilprobleme der Größe $\frac{n}{b}$ aufteilt und das rekursiv
 - $f(n)$ beschreibt den zusätzlichen Aufwand des Zusammenfügens bzw. der Verarbeitung, der nicht zur rekursiven Arbeit dazugehört

- das Mastertheorem vergleicht dann die Gesamtarbeit aller rekursiven Aufrufe mit der zusätzlichen Arbeit $f(n)$
- abhängig vom Verhältnis liefert das Mastertheorem eine Komplexitätsklasse für die rekursive Gleichung $T(n)$
 1. Falls $a < b^d$, dann $T(n) \in O(n^d)$
 2. Falls $a = b^d$, dann $T(n) \in O(n^d \cdot \log(n))$
 3. Falls $a > b^d$, dann $T(n) \in O(n^{\log_b(a)})$
- wobei gelten muss: $a > 0, b > 1, f(n) \in O(n^d), d \geq 0$

Aufgabe 1

1 a)

$$T(n) = 8T\left(\frac{n}{2}\right) + n \cdot \log(n)$$

- $a = 8, b = 2$
- $f(n) = n \cdot \log(n) \in O(n^d)$ mit $1 < d < 2$ (weil $O(n \cdot \log(n)) < O(n^2)$)
- das heißt, $a > b^d \rightarrow T(n) \in O(n^{\log_2(8)}) = O(n^3)$

1 b)

$$T(n) = 7T\left(\frac{n}{3}\right) + n^2 + 4n + 1$$

- $a = 7, b = 3$
- $f(n) = n^2 + 4n + 1 \in O(n^2) \rightarrow d = 2$
- das heißt, $a < b^d \rightarrow T(n) \in O(n^2)$

1 c)

$$T(n) = n \cdot T\left(\frac{n}{2}\right) + 1$$

- nicht lösbar, weil $a = n$

1 d)

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + n^3$$

- $a = 2, b = 2$
- $f(n) = n^3 \in O(n^3) \rightarrow d = 3$
- das heißt, $a < b^d \rightarrow T(n) \in O(n^3)$

1 e)

$$T(n) = T(\sqrt{n}) + 1$$

- um die Gleichung in die richtige Form zu bringen, muss substituiert werden:
- Substituiere mit $m := \log_2(n) \Leftrightarrow 2^m = n$

$$\rightarrow T(n) = T(2^m) = T(2^{\frac{m}{2}}) + 1$$

- Setze $S(m) := T(2^m)$
- Dadurch ergibt sich: $S(m) = T(2^{\frac{m}{2}}) + 1 = S(\frac{m}{2}) + 1$

$$\rightarrow S(m) = S(\frac{m}{2}) + 1$$

- $a = 1, b = 2$
- $f(m) = 1 \in O(1) \rightarrow d = 0$

- das heißt,
 $a = b^d \rightarrow S(m) \in O(m^0 \cdot \log(m)) = O(\log(m))$
- Rücksubstitution mit $m = \log_2(n)$:

$$S(m) = T(2^m) = T(n) \in O(\log(\log_2(n)))$$

1 f)

$$T(n) = a \cdot T(\sqrt[x]{n}) + \log_y(n)$$

- mit $x, y > 1$ und $\log_x(a) < 1$
- Substituieren mit $m = \log_y(n) \Leftrightarrow y^m = n$

$$\begin{aligned} \rightarrow T(n) &= T(y^m) = a \cdot T(\sqrt[x]{y^m}) + \log_y(y^m) \\ &= a \cdot T(y^{\frac{m}{x}}) + m \end{aligned}$$

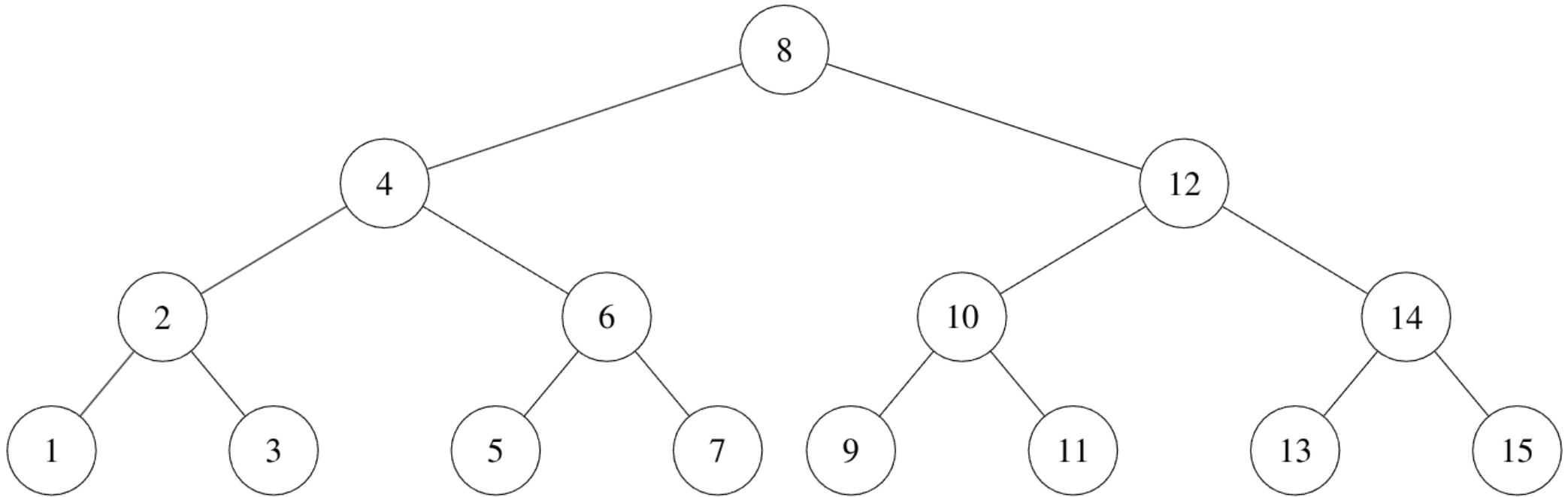
- Setze $S(m) = T(y^m)$
- Dadurch ergibt sich:
$$S(m) = a \cdot T(y^{\frac{m}{x}}) + m = a \cdot S(\frac{m}{x}) + m$$
$$\rightarrow S(m) = a \cdot S(\frac{m}{x}) + m$$
- $a = a, b = x$
- $f(n) = m \in O(1) \rightarrow d = 1$

- aus der Bedingung $\log_b(a) < 1$ ergibt sich
 $\log_x(a) < d \Leftrightarrow a < b^d \Rightarrow S(m) \in O(m^d) = O(m^1)$
- Rücksubstitution mit $m = \log_y(n)$:

$$S(m) = T(y^m) = T(n) \in O(\log_y(n))$$

Aufgabe 2

2 a)



Um welchen Baum handelt es sich?

- ausgeglichener Binärbaum
- außerdem ist er ein Binary Search Tree (kommt noch in der Vorlesung vor)

Welche Arität besitzt der Baum?

- Arität = 2, weil ein Knoten maximal 2 Kindknoten haben kann

2 b)

Welche Höhe hat der Baum?

- Höhe = 4

Wie viele Knoten?

- Da der Baum maximal ausgefüllt ist, kann die Formel $2^h - 1$ verwendet werden: $2^4 - 1 = 15$ Knoten
- Für die Anzahl der Blätter kann diese Formel 2^{h-1} verwendet werden: $2^{4-1} = 8$

2 c)

Ein Baum hat die Höhe 3 und er wird um zwei Ebenen erhöht.

Wie viele Knoten k sind mindestens dazu gekommen?

- 2

Wie viele Knoten k kann der Baum jetzt maximal haben?

- Dazu kann wieder die Formel $2^h - 1$ verwendet werden
- $2^5 - 1 = 31$

2 d)

Wie viele Knoten k hat ein Baum mindestens und maximal, wenn die Höhe h nun um n erhöht wird?

- mindestens: $h + n$
 - Ein Baum der Höhe $h + n$ hat mindestens $h + n$ Knoten
- maximal: $2^{h+n} - 1$

2 e)

Wie ändern sich die Formel in Abhängigkeit von der Höhe h aus der Vorlesung (Kapitel 1, Seite 56), wenn der Baum die Arität n hat?

- Minimale Knotenanzahl k_{min}
- Maximale Knotenanzahl k_{max}
- Maximale Anzahl innerer Knoten k_{in}
- Anzahl der Blätter b

Minimale Knotenanzahl k_{min} :

Ein Baum mit der Höhe h hat immer mindestens h Knoten:

$$k_{min} = h$$

Das trifft genau dann zu, wenn jeder Knoten genau ein Kind hat.

Maximale Knotenanzahl k_{max} :

- Die Funktion $2^h - 1 = k_{max}$ funktioniert nur bei Binärbäumen (Arität = 2).

- Um eine Funktion zu finden, die für eine Arität n gilt, schauen wir uns zunächst an wie sich die maximale Anzahl an Elementen in einem Binärbaum verändert, wenn die Höhe wächst:

$$h = 1 \rightarrow k_{max} = 1$$

$$h = 2 \rightarrow k_{max} = 3$$

$$h = 3 \rightarrow k_{max} = 7$$

$$h = 4 \rightarrow k_{max} = 15$$

- Der Abstand zwischen den Zahlen ist immer ein Vielfaches von 2 (2, 4, 8) und die 1 kann man auch als ein Vielfaches von 2 darstellen:

$$h = 1 \rightarrow k_{max} = 2^0 = 1$$

$$h = 2 \rightarrow k_{max} = 2^0 + 2^1 = 3$$

$$h = 3 \rightarrow k_{max} = 2^0 + 2^1 + 2^2 = 7$$

$$h = 4 \rightarrow k_{max} = 2^0 + 2^1 + 2^2 + 2^3 = 15$$

- k_{max} wird also aus der Summe aller Vielfachen der Arität n berechnet mit einer Potenz bis $h - 1$.
- Somit lässt sich also die Formel ableiten:

$$k_{max} = \sum_{i=0}^{h-1} n^i$$

- Wenn die Summe mit $\frac{n-1}{n-1}$ multipliziert wird, kann sie auf folgende Form gekürzt werden: $k_{max} = \sum_{i=0}^{h-1} n^i = \frac{n^h - 1}{n - 1}$

Maximale Anzahl innerer Knoten k_{in} :

Da die maximale Anzahl der inneren Knoten k_{in} bei einem Baum der Höhe h gleich der maximalen Anzahl an Knoten k_{max} in einem Baum der Höhe $h - 1$ ist, muss die Formel für die maximale Knotenanzahl nur leicht verändert werden, um die Anzahl der inneren Knoten zu bekommen:

$$k_{max} = \frac{n^h - 1}{n - 1} \quad \rightarrow \quad k_{in} = \frac{n^{h-1} - 1}{n - 1}$$

Anzahl der Blätter b :

Um die Anzahl der Blätter b eines Baumes mit der Arität n zu berechnen, kann folgenden Formel verwendet werden:

$$b = n^{h-1}$$

2 f)

Warum hat die Suche eines Elements in einem Baum eine lineare Laufzeit?

- Um ein Element in einem Baum zu finden, müssen zwei Schritte durchgeführt werden:
 1. Es wird durch den Baum traversiert (Inorder, Preorder, Postorder) und das Ergebnis in einer Liste gespeichert.
 - die Dauer der Traversierung hängt von der Anzahl der Elemente k ab (lineare Laufzeit $O(k)$)
 2. Es wird über die Liste iteriert und geprüft, ob das gesuchte Element mit dem aktuellen Element übereinstimmt.
 - im besten Fall braucht es hierbei nur eine Operation und im schlimmsten Fall k Operationen

- Um die durchschnittliche Anzahl an Operationen zu berechnen, addiert man die Anzahl der Operationen aller Möglichkeiten zusammen und teilt die Summe durch die Anzahl der Möglichkeiten:

$$\frac{1}{k} \sum_{i=1}^k i = \frac{1}{k} \cdot \frac{(k+1)k}{2} = \frac{k+1}{2}$$

- Die Summe hier wurde mit Hilfe der Gaußschen Summenformel umgeformt

2 g)

- 1. Wie kann ein Algorithmus aussehen, um ein Element in einem geordneten Baum zu suchen?**
- 2. Welche Laufzeit hat dieser Algorithmus?**
- 3. Wie ändert sich die Laufzeit bei einer Arität von n ?**

Frage 1: Algorithmus

1. Der Algorithmus beginnt bei der Wurzel. Ist das Element in der Wurzel das gesuchte Element? Wenn ja: Fertig!
2. Ist das Element in der Wurzel kleiner als das gesuchte Element?
3. Wenn ja, überprüfe, ob es einen rechten Teilbaum gibt. Wenn nein, überprüfe, ob es einen linken Teilbaum gibt.
 - Wenn es den jeweiligen Teilbaum nicht gibt, konnte das Element nicht gefunden werden.
4. Wenn es den jeweiligen Teilbaum gibt, gehe zur Wurzel dieses Teilbaums und starte von Schritt 1.

Frage 2: Laufzeit

- Bei jedem Element, das untersucht wird, führt der Algorithmus maximal 3 Schritte durch:
 1. Ist das Element das gesuchte Element?
 2. Ist es kleiner als das gesuchte Element?
 3. Existiert der jeweilige Teilbaum?

- Das bedeutet, dass eine Iteration des Algorithmus eine Komplexität von $O(3)$ hat.
- Der Algorithmus wird maximal so oft durchlaufen, wie der Baum hoch ist, also $h = \log_2(k)$ mal (siehe Übungsfolien 2, 1aa)).
- Da gilt $O(3) \subset O(\log_2(k))$, ist die Komplexität des Algorithmus $O(\log_2(k))$

Frage 3: Laufzeit bei Arität n

- Anstatt maximal 3 Schritte bei jedem Element, das untersucht wird, durchzuführen, muss der Algorithmus bei einer Arität n jetzt $n + 2$ Schritte durchführen:
 - die $+2$ sind die ersten zwei Schritte, bei denen das momentane Element mit dem gesuchten Element verglichen wird und ob es kleiner als das gesuchte ist
 - das n steht für die n Teilbäume, die ein Knoten haben kann (Arität)
- dadurch ergibt sich eine Komplexität von $O(n)$ für eine Iteration

- bei maximal $\log_2(k)$ Iterationen hat der Algorithmus eine Komplexität von $O(n \cdot \log_2(k))$
- Wenn die Arität n größer ist als die Anzahl der Knoten k , dann sind alle Knoten bei der Wurzel und dann ist der Baum gleichzeitig eine Liste
 - in der Praxis werden Bäume mit einer größeren Arität als die Knotenanzahl nicht verwendet

Aufgabe 3

Algorithmus 1

```
1 public static int[] zweiZahlenSumme(int[] array, int zielZahl) {
2     for (int i = 0; i < array.length - 1; i++) {
3         int ersteZahl = array[i];
4         for (int j = i + 1; j < array.length; j++) {
5             int zweiteZahl = array[j];
6             if (ersteZahl + zweiteZahl == zielZahl) {
7                 return new int[] { ersteZahl, zweiteZahl };
8             }
9         }
10    }
11    return new int[0];
12 }
```

Was macht der Algorithmus?

- Der Algorithmus iteriert mit der ersten Schleife durch das Array bis zur vorletzten Stelle (`array.length - 1`) und holt sich dabei die erste Zahl.
- Mit der zweiten Schleife iteriert er auch durch das Array und holt sich die restlichen Zahlen, um sie mit der ersten Zahl zu vergleichen.
- Falls die Summe beider Zahlen das Ergebnis ist, gibt er ein Array mit beiden Zahlen zurück und falls kein Ergebnis gefunden wurde, gibt er ein leeres Array zurück.

Komplexität

- beide for-Schleifen haben zusammen $O(n^2)$
- alle anderen Zeilen haben $O(1)$
- Komplexitätsklasse: $O(n^2)$

Algorithmus 2

```
1 public static int[] zweiZahlenSumme2(int[] array, int zielZahl) {  
2     Arrays.sort(array);  
3     int links = 0;  
4     int rechts = array.length - 1;  
5     while (links < rechts) {  
6         int aktuelleZahl = array[links] + array[rechts];  
7         if (aktuelleZahl == zielZahl) {  
8             return new int[] { array[links], array[rechts] };  
9         } else if (aktuelleZahl < zielZahl) {  
10             links++;  
11         } else if (aktuelleZahl > zielZahl) {  
12             rechts--;  
13         }  
14     }  
15     return new int[0];  
16 }
```

Was macht der Algorithmus?

- Das Array wird sortiert.
- Der Algorithmus nimmt sich dann die kleinste und die größte Zahl und addiert sie zusammen.

- Wenn das Ergebnis nicht der gesuchten Zahl entspricht, schaut er, ob das Ergebnis größer oder kleiner ist.
- Je nachdem nimmt er sich dann die entweder die nächst größere oder nächst kleinere Zahl, bis er das richtige Ergebnis gefunden hat.

Komplexität

- `Arrays.sort()` hat $O(n \cdot \log(n))$
- while-Schleife hat $O(n)$
- alle anderen Zeilen haben $O(1)$
- Komplexitätsklasse: $O(n \cdot \log(n))$

Algorithmus 3

```
1 public static int[] zweiZahlenSumme3(int[] array, int zielZahl) {  
2     Set<Integer> nums = new HashSet<>();  
3     for (int num : array) {  
4         int treffer = zielZahl - num;  
5         if (nums.contains(treffer)) {  
6             return new int[] {  
7                 treffer, num  
8             };  
9         } else {  
10            nums.add(num);  
11        }  
12    }  
13    return new int[0];  
14 }
```

Was macht der Algorithmus?

- Der Algorithmus erstellt eine Menge (Set).
- Dort speichert er alle Zahlen, die er sich bereits angeschaut hat.
- Als nächstes iteriert er durch das Array und zieht die aktuelle Zahl vom gesuchten Ergebnis ab.

- Wenn das Ergebnis nicht in der Menge ist, fügt er die aktuelle Zahl der Menge zu.
- Somit kommen immer mehr Zahlen aus dem Array in die Menge, bis der Algorithmus irgendwann eine Zahl aus dem Array von der gesuchten Zahl abzieht und das Ergebnis eine Zahl ist, die sich bereits in der Menge befindet.

Komplexität

- for-Schleife hat $O(n)$
- alle anderen Zeilen haben $O(1)$
- Komplexitätsklasse: $O(n)$

Welcher Algorithmus?

Unter welchen Umständen würde man welchen Algorithmus verwenden?

- Wenn nur die Laufzeit wichtig ist, ist der Algorithmus 3 am besten.
- Wenn der benötigte Speicherplatz aber eine Rolle spielt, hat Algorithmus 3 eine schlechtere Platzkomplexität von $O(n)$ (wegen der Menge), während die anderen jeweils eine bessere Platzkomplexität von $O(1)$ haben.